Multi-Vendor Integration Pattern — The Shape, and Where It Drifts

InstantMudra Enterprises Admin Portal

The intended 3-layer pattern — worked example: AEPS merchant onboarding

AepsVendorService (interface)
one impl per vendor, e.g.
FingPayAepsServiceImpl

→

AepsVendorFactory
exact key match, falls back to
substring key.contains(code)

→

AepsOrchestrator
routing → credentials → vendor bean → call
— but only 82 lines, ONE method: registerMerchant()

VendorRouting — config model (per sub-product)

routingPriority1 / 2 / 3 (Vendors FK) — CommonMethods.getVendors() returns the first ACTIVE priority slot, in order
Also stores volume1-3 / value1-3 / volumeNumberOfTxn / volumeValueOfTxn / valueValueNumberOfTxn / valueValueOfTxn — columns shaped for percentage/threshold-based traffic splitting across vendors
⚠ getVendors() never reads any of the volume/value columns — only simple priority failover is actually implemented

AEPS / DMT / Recharge / Payout factories
fallback match: registeredKey.contains(requestedCode)
— unordered HashMap iteration if multiple keys match

PaymentGatewayInitiatorFactory
fallback match: first whitespace-separated word only
— a completely different, independently-invented strategy

Where the pattern drifts — three different shapes, all verified by line count

AEPS — total bypass

AepsOrchestrator: **82 lines**, one method — merchant onboarding only.
InstantpayAepsTransactionServiceImpl: **1,171 lines** — balance inquiry, cash withdrawal, mini-statement, ALL of it — resolves its own vendor routing independently, never calls AepsVendorFactory or AepsOrchestrator.
14× the size of the "official" orchestrator, and it's the code path actually doing the work.

DMT — parallel duplication

DmtOrchestrator: **2,207 lines** — large, and genuinely the active, used orchestration path.
RemitterServiceImpl (**936 lines**) + DmtBeneficiaryServiceImpl (**829 lines**) — older, Paysprint-hardcoded implementations that run **alongside** the orchestrator, not replaced by it.
Two live code paths for the same domain — which one actually handles a given request depends on which controller/ endpoint was called, not on any single source of truth.

BBPS — hybrid: looks up config, then ignores it

BBPSServiceImpl (759 lines) DOES query VendorRoutingRepository.findBySubProductId() — so it looks like config-driven routing — but then hardcodes the literal string "A2Z" into the transaction record and description regardless of what the routing actually resolved to. RechargeVendorFactory is never called from this file.

Non-functional stub vendors — not even registered as Spring beans

AirtelAepsServiceImpl, A2ZRechargeServiceImpl, FingpayDmtServiceImpl — each implements its vendor interface but carries **no @Service / @Component annotation**, so Spring never instantiates them — they're absent from the factory's constructor-injected List<VendorService> entirely, on top of every method already returning null/empty. Routing to "airtel" / "a2z" / "fingpay" fails at the factory lookup, before any stub method would even run.

Standard / as-designed · Correctly-routed, factory/orchestrator path · Flagged drift / dead config · Severe drift — pattern totally bypassed